Análisis de Caso: Preprocesamiento y Escalamiento de Datos

Empresa: RetailData Analytics

Especialista en Preparación de Datos: Camila Garrido
Tecnologías: Python 3.13.5, Pandas, Scikit-Learn
Repositorio GitHub: https://github.com/CamilaGarrido/retail-data-preprocessing.git

Introducción

Este proyecto forma parte de una iniciativa de RetailData Analytics para una cadena de supermercados.

El objetivo principal es transformar un conjunto de datos demográficos y de comportamiento del cliente para identificar perfiles propensos a realizar compras recurrentes.

A continuación, se detalla el proceso de:

- **Limpieza de datos** (Imputación de valores nulos).
- **Codificación de variables categóricas** (Label y One-Hot Encoding).
- **Escalamiento de variables numéricas** (Normalización y Estandarización).

---

1. Inicialización y Carga de Datos

Como Especialista en Preparación de Datos , mi primera tarea es cargar el fragmento de datos demográficos proporcionado por el cliente para identificar problemas de calidad como valores nulos y escalas inconsistentes.

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, MinMaxScaler, StandardScaler

# Recreación del fragmento de datos proporcionado
data = {
    'ID': [1, 2, 3, 4],
    'Edad': [25, 45, 30, 40],
    'Ciudad': ['Madrid', 'Sevilla', 'Madrid', 'Barcelona'],
    'Ingresos': [30000, 50000, np.nan, 40000]
}

df = pd.DataFrame(data)
print("Dataset original recibido:")
df
```

```
Dataset original recibido:
```

Out[1]:

	ID	Edad	Ciudad	Ingresos
0	1	25	Madrid	30000.0
1	2	45	Sevilla	50000.0
2	3	30	Madrid	NaN
3	4	40	Barcelona	40000.0

2. Imputación de Valores Faltantes

El análisis inicial muestra un valor NaN en la columna Ingresos. Procederé a imputar este valor utilizando la media de la columna.

In [2]:
```
# Requisito: Imputar el valor faltante en Ingresos con la media
imputer = SimpleImputer(strategy='mean')
df['Ingresos'] = imputer.fit_transform(df[['Ingresos']])
```

```
print("Dataset después de la imputación:")
df
```

```
Dataset después de la imputación:
```

Out[2]:

	ID	Edad	Ciudad	Ingresos
0	1	25	Madrid	30000.0
1	2	45	Sevilla	50000.0
2	3	30	Madrid	40000.0
3	4	40	Barcelona	40000.0

3. Codificación de Variables Categóricas

Para que el modelo predictivo pueda procesar la columna Ciudad, aplicaremos tres técnicas de codificación: Label Encoding, One-Hot Encoding y Variables Dummy.

In [3]:

```
# A. Label Encoding: Asigna un número entero a cada categoría
le = LabelEncoder()
df['Ciudad_Label'] = le.fit_transform(df['Ciudad'])

# B. One-Hot Encoding: Crea columnas binarias independientes
# Nota: Usamos drop='first' opcionalmente para evitar la multicolinealidad
ohe = OneHotEncoder(sparse_output=False)
ciudad_ohe = ohe.fit_transform(df[['Ciudad']])

# C. Variables Dummy: Método directo de Pandas
df_final = pd.get_dummies(df, columns=['Ciudad'], prefix='City')

print("Dataset con variables categóricas codificadas:")
df_final
```

```
Dataset con variables categóricas codificadas:
```

Out[3]:

	ID	Edad	Ingresos	Ciudad_Label	City_Barcelona	City_Madrid	City_Sevilla
0	1	25	30000.0	1	False	True	False
1	2	45	50000.0	2	False	False	True
2	3	30	40000.0	1	False	True	False
3	4	40	40000.0	0	True	False	False

4. Escalamiento de Datos

Las variables Edad e Ingresos poseen escalas muy distintas. Aplicaremos Normalización Min-Max y Estandarización Z-Score para homogeneizar los rangos.

In [4]:

```
# Definición de las columnas numéricas a escalar
cols_to_scale = ['Edad', 'Ingresos']

# Normalización Min-Max: Escala los datos al rango [0, 1]
scaler_minmax = MinMaxScaler()
df_final[['Edad_MinMax', 'Ing_MinMax']] = scaler_minmax.fit_transform(df_final[cols_to_scale])

# Estandarización Z-Score: Escala con media 0 y desviación estándar 1
scaler_zscore = StandardScaler()
df_final[['Edad_Z', 'Ing_Z']] = scaler_zscore.fit_transform(df_final[cols_to_scale])

print("Visualización final de los datos preprocesados y escalados:")
df_final
```

```
Visualización final de los datos preprocesados y escalados:
```

Out[4]:

	ID	Edad	Ingresos	Ciudad_Label	City_Barcelona	City_Madrid	City_Sevilla	Edad_MinMax	Ing_MinMax	Edad_Z	Ing_Z
0	1	25	30000.0	1	False	True	False	0.00	0.0	-1.264911	-1.414214
1	2	45	50000.0	2	False	False	True	1.00	1.0	1.264911	1.414214
2	3	30	40000.0	1	False	True	False	0.25	0.5	-0.632456	0.000000
3	4	40	40000.0	0	True	False	False	0.75	0.5	0.632456	0.000000

In [5]:

```
# Guardar el conjunto de datos transformado
df_final.to_csv('RetailData_Transformado.csv', index=False)
print("Archivo CSV generado con éxito.")
```

```
Archivo CSV generado con éxito.
```

Respuestas de Reflexión

¿Por qué es importante realizar estas tareas antes de entrenar un modelo de Machine Learning?

1. Calidad de datos: Los algoritmos no pueden manejar valores nulos (NaN) sin que se produzcan errores.
2. Compatibilidad: Los modelos matemáticos requieren entradas numéricas; por ello, las variables categóricas deben codificarse.
3. Equilibrio de magnitudes: Si no se escalan las variables, aquellas con rangos numéricos mayores (como Ingresos) podrían dominar el entrenamiento, haciendo que el modelo ignore variables con rangos menores (como Edad).

¿Qué diferencias observaste entre la normalización y la estandarización?

1. Normalización (Min-Max): Comprime los datos dentro de un rango estricto (usualmente de 0 a 1). Es útil cuando la distribución de los datos no es normal.
2. Estandarización (Z-Score): No tiene un rango fijo, sino que centra los datos en una media de 0. Es mucho más robusta ante valores atípicos (outliers) que podrían distorsionar la normalización.